

UNIT – 2

CONCURRENT PROCESSES

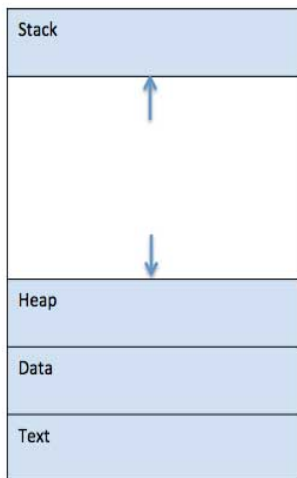
Process

A process is basically a program in execution. The execution of a process must progress in a sequential fashion. a process is defined as an entity which represents the basic unit of work to be implemented in the system. we write our computer programs in a text file and when we execute this program, it becomes a process which performs all the tasks mentioned in the program.

When a program is loaded into the memory and it becomes a process, it can be divided into four sections — stack, heap, text and data. The following image shows a simplified layout of a process inside main memory —

Components of a Process

It is divided into the following four sections:



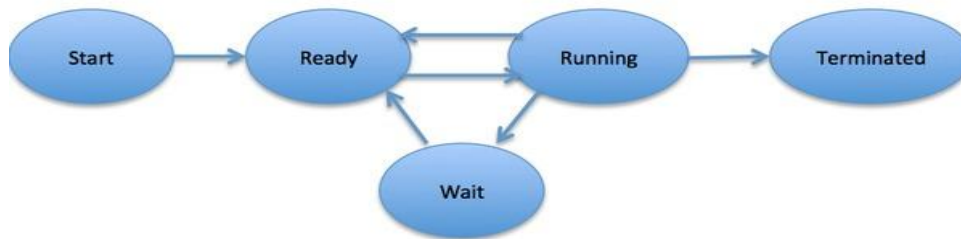
S.N.	Component & Description
1	Stack: The process Stack contains the temporary data such as method/function parameters, return address and local variables.
2	Heap: This is dynamically allocated memory to a process during its run time.
3	Text: This includes the current activity represented by the value of Program Counter and the contents of the processor's registers.
4	Data: This section contains the global and static variables.

Process Life Cycle

When a process executes, it passes through different states. These stages may differ in different operating systems, and the names of these states are also not standardized.

In general, a process can have one of the following five states at a time.

S.N.	State & Description
1	Start: This is the initial state when a process is first started/created.
2	Ready: The process is waiting to be assigned to a processor. Ready processes are waiting to have the processor allocated to them by the operating system so that they can run. Process may come into this state after Start state or while running it by but interrupted by the scheduler to assign CPU to some other process.
3	Running: Once the process has been assigned to a processor by the OS scheduler, the process state is set to running and the processor executes its instructions.
4	Waiting: Process moves into the waiting state if it needs to wait for a resource, such as waiting for user input, or waiting for a file to become available.
5	Terminated or Exit: Once the process finishes its execution, or it is terminated by the operating system, it is moved to the terminated state where it waits to be removed from main memory.



Process Control Block (PCB)

A Process Control Block is a data structure maintained by the Operating System for every process. The PCB is identified by an integer process ID (PID). A PCB keeps all the information needed to keep track of a process as listed below in the table –

S.N.	Information & Description
1	Process State: The current state of the process i.e., whether it is ready, running, waiting, or whatever.
2	Process privileges: This is required to allow/disallow access to system resources.
3	Process ID: Unique identification for each of the process in the operating system.
4	Pointer: A pointer to parent process.
5	Program Counter: Program Counter is a pointer to the address of the next instruction to be executed for this process.
6	CPU registers: Various CPU registers where process need to be stored for execution for running state.
7	CPU Scheduling Information: Process priority and other scheduling information which is required to schedule the process.
8	Memory management information: This includes the information of page table, memory limits, Segment table depending on memory used by the operating system.
9	Accounting information: This includes the amount of CPU used for process execution, time limits, execution ID etc.
10	IO status information: This includes a list of I/O devices allocated to the process.

The architecture of a PCB is completely dependent on Operating System and may contain different information in different operating systems. Here is a simplified diagram of a PCB –

Process ID
State
Pointer
Priority
Program counter
CPU registers
I/O information
Accounting information
etc....

The PCB is maintained for a process throughout its lifetime, and is deleted once the process terminates.

Process Synchronization

Process Synchronization is the task of coordinating the execution of processes in a way that no two processes can have access to the same shared data and resources.

It is specially needed in a multi-process system when multiple processes are running together, and more than one processes try to gain access to the same shared resource or data at the same time.

This can lead to the inconsistency of shared data. So the change made by one process not necessarily reflected when other processes accessed the same shared data. To avoid this type of inconsistency of data, the processes need to be synchronized with each other.

Concurrency

Concurrency is the execution of the multiple instruction sequences at the same time. It happens in the operating system when there are several process threads running in parallel. The running process threads always communicate with each other through shared memory or message passing. Concurrency results in sharing of resources result in problems like deadlocks and resources starvation.

It helps in techniques like coordinating execution of processes, memory allocation and execution scheduling for maximizing throughput.

Principles of Concurrency:

Both interleaved and overlapped processes can be viewed as examples of concurrent processes, they both present the same problems. The relative speed of execution cannot be predicted. It depends on the following:

- The activities of other processes
- The way operating system handles interrupts
- The scheduling policies of the operating system

Problems in Concurrency :

- **Sharing global resources** – Sharing of global resources safely is difficult. If two processes both make use of a global variable and both perform read and write on that variable, then the order in which various read and write are executed is critical.
- **Optimal allocation of resources** – It is difficult for the operating system to manage the allocation of resources optimally.
- **Locating programming errors** – It is very difficult to locate a programming error because reports are usually not reproducible.
- **Locking the channel** – It may be inefficient for the operating system to simply lock the channel and prevents its use by other processes.

Advantages of Concurrency :

- **Running of multiple applications** – It enable to run multiple applications at the same time.
- **Better resource utilization** – It enables that the resources that are unused by one application can be used for other applications.
- **Better average response time** – Without concurrency, each application has to be run to completion before the next one can be run.
- **Better performance** – It enables the better performance by the operating system. When one application uses only the processor and another application uses only the disk drive then the time to run both applications concurrently to completion will be shorter than the time to run each application consecutively.

Drawbacks of Concurrency :

- It is required to protect multiple applications from one another.
- It is required to coordinate multiple applications through additional mechanisms.
- Additional performance overheads and complexities in operating systems are required for switching among applications.
- Sometimes running too many applications concurrently leads to severely degraded performance.

Issues of Concurrency :

- **Non-atomic** – Operations that are non-atomic but interruptible by multiple processes can cause problems.
- **Race conditions** – A race condition occurs if the outcome depends on which of several processes gets to a point first.
- **Blocking** – Processes can block waiting for resources. A process could be blocked for long period of time waiting for input from a terminal. If the process is required to periodically update some data, this would be very undesirable.
- **Starvation** – It occurs when a process does not obtain service to progress.
- **Deadlock** – It occurs when two processes are blocked and hence neither can proceed to execute.

Producer-Consumer problem

The Producer-Consumer problem is a classic problem this is used for multi-process synchronization i.e. synchronization between more than one processes.

In the producer-consumer problem, there is one Producer that is producing something and there is one Consumer that is consuming the products produced by the Producer. The producers and consumers share the same memory buffer that is of fixed-size.

The job of the Producer is to generate the data, put it into the buffer, and again start generating data. While the job of the Consumer is to consume the data from the buffer.

What's the problem here?

The following are the problems that might occur in the Producer-Consumer:

- The producer should produce data only when the buffer is not full. If the buffer is full, then the producer shouldn't be allowed to put any data into the buffer.
- The consumer should consume data only when the buffer is not empty. If the buffer is empty, then the consumer shouldn't be allowed to take any data from the buffer.
- The producer and consumer should not access the buffer at the same time.

What's the solution?

The above three problems can be solved with the help of semaphores

Semaphores : A semaphore is an integer variable, shared among multiple processes. The main aim of using a semaphore is process synchronization and access control for a common resource in a concurrent environment.

A semaphore has two indivisible (atomic) operations, namely: Wait and Signal .

The definitions of wait and signal are as follows –

- **Wait:** The wait operation decrements the value of its argument S, if it is positive. If S is negative or zero, then no operation is performed.

```
wait(S)
{
    while (S<=0);

    S--;
}
```

- **Signal** : The signal operation increments the value of its argument S.

```
signal(S)
{
    S++;
}
```

Types of Semaphores

There are two main types of semaphores i.e. counting semaphores and binary semaphores.

- **Counting Semaphores**

These are integer value semaphores and have an unrestricted value domain. These semaphores are used to coordinate the resource access, where the semaphore count is the number of available resources. If the resources are added, semaphore count automatically incremented and if the resources are removed, the count is decremented.

- **Binary Semaphores**

The binary semaphores are like counting semaphores but their value is restricted to 0 and 1. The wait operation only works when the semaphore is 1 and the signal operation succeeds when semaphore is 0. It is sometimes easier to implement binary semaphores than counting semaphores.

Advantages of Semaphores

Some of the advantages of semaphores are as follows –

- Semaphores allow only one process into the critical section. They follow the mutual exclusion principle strictly and are much more efficient than some other methods of synchronization.
- There is no resource wastage because of busy waiting in semaphores as processor time is not wasted unnecessarily to check if a condition is fulfilled to allow a process to access the critical section.
- Semaphores are implemented in the machine independent code of the microkernel. So they are machine independent.

Disadvantages of Semaphores

Some of the disadvantages of semaphores are as follows –

- Semaphores are complicated so the wait and signal operations must be implemented in the correct order to prevent deadlocks.
- Semaphores are impractical for large scale use as their use leads to loss of modularity. This happens because the wait and signal operations prevent the creation of a structured layout for the system.
- Semaphores may lead to a priority inversion where low priority processes may access the critical section first and high priority processes later.

In the producer-consumer problem, we use three semaphore variables:

1. **Semaphore S:** This semaphore variable is used to achieve mutual exclusion between processes. By using this variable, either Producer or Consumer will be allowed to use or access the shared buffer at a particular time. This variable is set to 1 initially.
2. **Semaphore E:** This semaphore variable is used to define the empty space in the buffer. Initially, it is set to the whole space of the buffer i.e. "n" because the buffer is initially empty.
3. **Semaphore F:** This semaphore variable is used to define the space that is filled by the producer. Initially, it is set to "0" because there is no space filled by the producer initially.

By using the above three semaphore variables and by using the *wait()* and *signal()* function, we can solve our problem(the *wait()* function decreases the semaphore variable by 1 and the *signal()* function increases the semaphore variable by 1).

The following is the pseudo-code for the producer:

```
void producer() {  
    while(T) {  
        produce()  
        wait(E)  
        wait(S)  
        append()  
        signal(S)  
        signal(F)  
    }  
}
```

The above code can be summarized as:

- *while()* is used to produce data, again and again, if it wishes to produce, again and again.
- *produce()* function is called to produce data by the producer.
- *wait(E)* will reduce the value of the semaphore variable "E" by one i.e. when the producer produces something then there is a decrease in the value of the empty space in the buffer. If the buffer is full i.e. the value of the semaphore variable "E" is "0", then the program will stop its execution and no production will be done.
- *wait(S)* is used to set the semaphore variable "S" to "0" so that no other process can enter into the critical section.
- *append()* function is used to append the newly produced data in the buffer.
- *signal(s)* is used to set the semaphore variable "S" to "1" so that other processes can come into the critical section now because the production is done and the append operation is also done.
- *signal(F)* is used to increase the semaphore variable "F" by one because after adding the data into the buffer, one space is filled in the buffer and the variable "F" must be updated.

This is how we solve the produce part of the producer-consumer problem. Now, let's see the consumer solution. The following is the code for the consumer:

```
void consumer() {  
    while(T) {  
        wait(F)  
        wait(S)  
        take()  
        signal(S)
```

```

    signal(E)
    use()
}
}

```

The above code can be summarized as:

- *while()* is used to consume data, again and again, if it wishes to consume, again and again.
- *wait(F)* is used to decrease the semaphore variable "F" by one because if some data is consumed by the consumer then the variable "F" must be decreased by one.
- *wait(S)* is used to set the semaphore variable "S" to "0" so that no other process can enter into the critical section.
- *take()* function is used to take the data from the buffer by the consumer.
- *signal(S)* is used to set the semaphore variable "S" to "1" so that other processes can come into the critical section now because the consumption is done and the take operation is also done.
- *signal(E)* is used to increase the semaphore variable "E" by one because after taking the data from the buffer, one space is freed from the buffer and the variable "E" must be increased.
- *use()* is a function that is used to use the data taken from the buffer by the process to do some operation.

So, this is how we can solve the producer-consumer problem.

Race condition

A race condition is an undesirable situation that occurs when a device or system attempts to perform two or more operations at the same time, but because of the nature of the device or system, the operations must be done in the proper sequence to be done correctly.

Race conditions are most commonly associated with computer memory or storage, a race condition may occur if commands to read and write a large amount of data are received at almost the same instant, and the machine attempts to overwrite some or all of the old data while that old data is still being read. The result may be one or more of the following: a computer crash, an "illegal operation," notification and shutdown of the program, errors reading the old data or errors writing the new data. A race condition can also occur if instructions are processed in the incorrect order.

Suppose for a moment that two processes need to perform a bit flip at a specific memory location. Under normal circumstances the operation should work like this:

Process 1	Process 2	Memory Value
-----------	-----------	--------------

Read value		0
Flip value		1
	Read value	1
	Flip value	0

In this example, Process 1 performs a bit flip, changing the memory value from 0 to 1. Process 2 then performs a bit flip and changes the memory value from 1 to 0.

If a race condition occurred causing these two processes to overlap, the sequence could potentially look more like this:

Process 1	Process 2	Memory Value
Read value		0
	Read value	0
Flip value		1
	Flip value	1

In this example, the bit has an ending value of 1 when its value should be 0. This occurs because Process 2 is unaware that Process 1 is performing a simultaneous bit flip.

Security vulnerabilities caused by race conditions

When a program that is designed to handle tasks in a specific sequence is asked to perform two or more operations simultaneously, an attacker can take advantage of the time gap between when the service is initiated and when a security control takes effect in order to create a deadlock or thread block situation

Preventing race conditions

To prevent such a race condition from developing, a priority scheme must be devised.

Mutual exclusion;

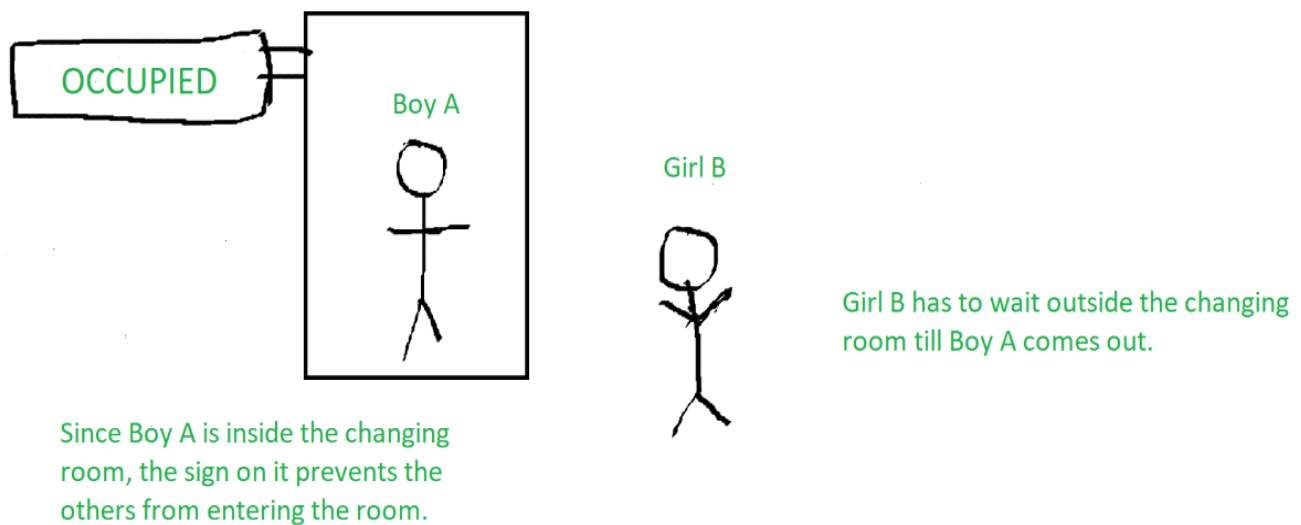
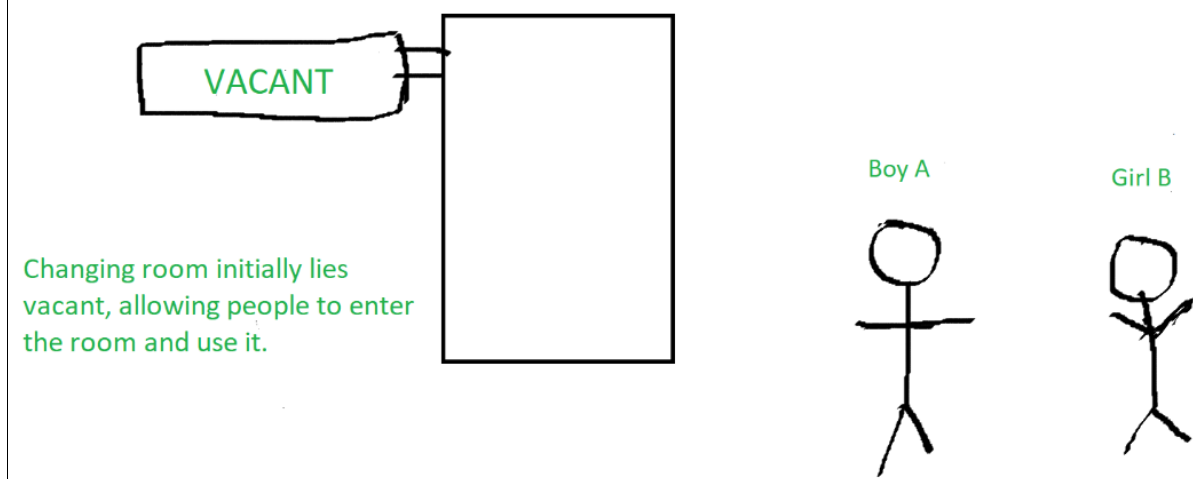
Mutual exclusion is a property of process synchronization which states that “no two processes can exist in the critical section at any given point of time”. The term was first coined by Dijkstra. Any process

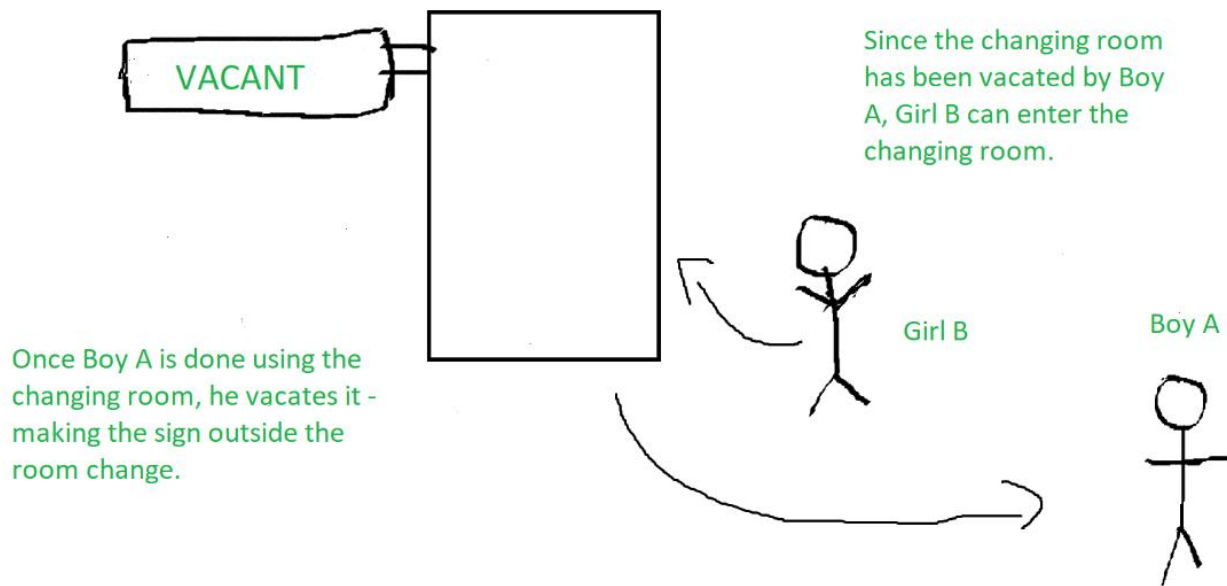
synchronization technique being used must satisfy the property of mutual exclusion, without which it would not be possible to get rid of a race condition.

To understand mutual exclusion, let's take an example.

Example:

In the clothes section of a supermarket, two people are shopping for clothes.





Mutual Exclusion Conditions

If you could arrange matters such that no two processes were ever in their critical sections simultaneously, you could avoid race conditions. You need four conditions to hold to have a good solution for the critical section problem (mutual exclusion).

1. No two processes may at the same moment inside their critical sections.
2. No assumptions are made about relative speeds of processes or number of CPUs.
3. No process outside its critical section should block other processes.
4. No process should wait arbitrary long to enter its critical section.

The critical section problem

Consider a system, assume that it consists of n processes. Each process having a segment of code. This segment of code is said to be critical section.

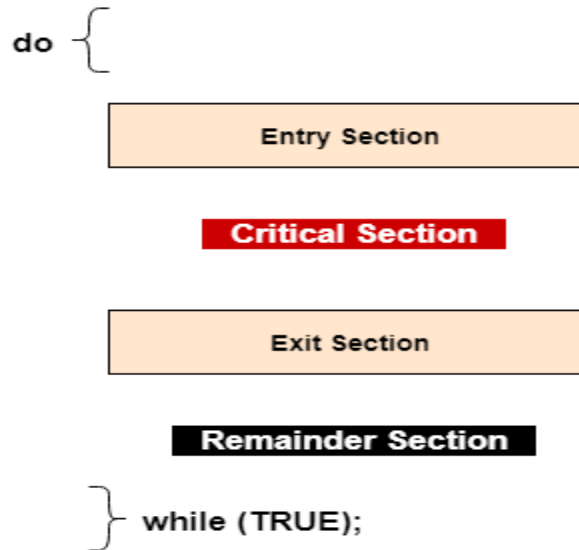
Problem – ensure that when one process is executing in its critical section, no other process is allowed to execute in its critical section.

Example: Railway Reservation System.

Two persons from different stations want to reserve their tickets, the train number, destination is common, the two persons try to get the reservation at the same time. Unfortunately, the available berths are only one; both are trying for that berth.

It is also called the critical section problem. Solution is when one process is executing in its critical section, no other process is to be allowed to execute in its critical section.

The critical section problem is to design a protocol that the processes can use to cooperate. Each process must request permission to enter its critical section. The section of code implementing this request is the **entry section**. The critical section may be followed by an **exit section**. The remaining code is the **remainder section**.



Sections of a Program

Here, are four essential elements of the critical section:

- **Entry Section:** It is part of the process which decides the entry of a particular process.
- **Critical Section:** This part allows one process to enter and modify the shared variable.
- **Exit Section:** Exit section allows the other process that are waiting in the Entry Section, to enter into the Critical Sections. It also checks that a process that finished its execution should be removed through this Section.
- **Remainder Section:** All other parts of the Code, which is not in Critical, Entry, and Exit Section, are known as the Remainder Section.

A solution to the critical section problem must satisfy the following 3 requirements:

1. **Mutual Exclusion.** If process P_i is executing in its critical section, then no other processes can be executing in their critical sections.
2. **Progress.** If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely.
3. **Bounded Waiting.** A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

Dekker's algorithm

Dekker's algorithm is the first solution of critical section problem. There are many versions of this algorithms, the 5th or final version satisfies the all the conditions below and is the most efficient among all of them.

The solution to critical section problem must ensure the following three conditions:

- Mutual Exclusion
- Progress

- Bounded Waiting

First version

- Dekker's algorithm succeeds to achieve mutual exclusion.
- It uses variables to control thread execution.
- It constantly checks whether critical section available.

Example

```
main(){
    int thread_no = 1;
    startThreads();
}

Thread1(){
    do {
        // entry section
        // wait until threadno is 1
        while (threadno == 2)
            ;
        // critical section
        // exit section
        // give access to the other thread
        threadno = 2;
        // remainder section
    } while (completed == false)
}

Thread2(){
    do {
        // entry section
        // wait until threadno is 2
        while (threadno == 1)
            ;
        // critical section
        // exit section
        // give access to the other thread
        threadno = 1;
        // remainder section
    } while (completed == false)
}
```

Problem

The problem of this first version of Dekker's algorithm is implementation of lockstep synchronization. It means each thread depends on other to complete its execution. If one of the two processes completes its execution, then the second process runs. Then it gives access to the completed one and waits for its run. But the completed one would never run and so it would never return access back to the second process. Thus the second process waits for infinite time.

Second Version

In Second version of Dekker's algorithm, lockstep synchronization is removed. It is done by using two flags to indicate its current status and updates them accordingly at the entry and exit section.

Example

```
main(){
    // flags to indicate whether each thread is in
    // its critical section or not.
    boolean th1 = false;
    boolean th2 = false;
    startThreads();
}

Thread1(){
    do {
        // entry section
        // wait until th2 is in its critical section
        while (th2 == true);
        // indicate thread1 entering its critical section
        th1 = true;
        // critical section
        // exit section
        // indicate th1 exiting its critical section
        th1 = false;
        // remainder section
    } while (completed == false)
}

Thread2(){
    do {
        // entry section
        // wait until th1 is in its critical section
        while (th1 == true);
        // indicate th2 entering its critical section
        th2 = true;
        // critical section
        // exit section
        // indicate th2 exiting its critical section
        th2 = false;
        // remainder section
    } while (completed == false)
}
```

Problem

Mutual exclusion is violated in this version. During flag update, if threads are preempted then both the threads enter into the critical section. Once the preempted thread is restarted, also the same can be observed at the start itself, when both the flags are false.

Third Version

In this version, critical section flag is set before entering critical section test to ensure mutual exclusion.

```
main(){
    // flags to indicate whether each thread is in
    // queue to enter its critical section
    boolean th1wantstoenter = false;
    boolean th2wantstoenter = false;
    startThreads();
}

Thread1(){
    do {
        th1wantstoenter = true;
        // entry section
        // wait until th2 wants to enter
        // its critical section
        while (th2wantstoenter == true)
            ;
        // critical section
        // exit section
        // indicate th1 has completed
        // its critical section
        th1wantstoenter = false;
        // remainder section
    } while (completed == false)
}

Thread2(){
    do {
        th2wantstoenter = true;
        // entry section
        // wait until th1 wants to enter
        // its critical section
        while (th1wantstoenter == true)
            ;
        // critical section
        // exit section
        // indicate th2 has completed
        // its critical section
        th2wantstoenter = false;
```

```

    // remainder section
} while (completed == false)
}

```

Problem

This version failed to solve the problem of mutual exclusion. It also introduces deadlock possibility, both threads could get flag simultaneously and they will wait for infinite time.

Fourth Version

In this version of Dekker's algorithm, it sets flag to false for small period of time to provide control and solves the problem of mutual exclusion and deadlock.

Example

```

main(){
    // flags to indicate whether each thread is in
    // queue to enter its critical section
    boolean th1wantstoenter = false;
    boolean th2wantstoenter = false;
    startThreads();
}

Thread1(){
    do {
        th1wantstoenter = true;
        while (th2wantstoenter == true) {
            // gives access to other thread
            // wait for random amount of time
            th1wantstoenter = false;
            th1wantstoenter = true;
        }
        // entry section
        // wait until th2 wants to enter
        // its critical section
        // critical section
        // exit section
        // indicate th1 has completed
        // its critical section
        th1wantstoenter = false;
        // remainder section
    } while (completed == false)
}

Thread2(){
    do {
        th2wantstoenter = true;
        while (th1wantstoenter == true) {

```



```

    // gives access to other thread
    // wait for random amount of time
    th2wantstoenter = false;
    th2wantstoenter = true;
}
// entry section
// wait until th1 wants to enter
// its critical section
// critical section
// exit section
// indicate th2 has completed
// its critical section
th2wantstoenter = false;
// remainder section
} while (completed == false)
}

```

Problem

Indefinite postponement is the problem of this version. Random amount of time is unpredictable depending upon the situation in which the algorithm is being implemented, hence it is not acceptable in case of business critical systems.

Fifth Version (Final Solution)

In this version, favored thread motion is used to determine entry to critical section. It provides mutual exclusion and avoiding deadlock, indefinite postponement or lockstep synchronization by resolving the conflict that which thread should execute first. This version of Dekker's algorithm provides the complete solution of critical section problems.

Example

```

main(){
    // to denote which thread will enter next
    int favouredthread = 1;
    // flags to indicate whether each thread is in
    // queue to enter its critical section
    boolean th1wantstoenter = false;
    boolean th2wantstoenter = false;
    startThreads();
}
Thread1(){
    do {
        thread1wantstoenter = true;
        // entry section
        // wait until th2 wants to enter
        // its critical section
    }
}

```

```

while (th2wantstoenter == true) {
    // if 2nd thread is more favored
    if (favaouredthread == 2) {
        // gives access to other thread
        th1wantstoenter = false;
        // wait until this thread is favored
        while (favouredthread == 2);
        th1wantstoenter = true;
    }
}
// critical section
// favor the 2nd thread
favouredthread = 2;
// exit section
// indicate th1 has completed
// its critical section
th1wantstoenter = false;
// remainder section
} while (completed == false)
}

Thread2(){
do {
    th2wantstoenter = true;
    // entry section
    // wait until th1 wants to enter
    // its critical section
    while (th1wantstoenter == true) {
        // if 1st thread is more favored
        if (favaouredthread == 1) {
            // gives access to other thread
            th2wantstoenter = false;
            // wait until this thread is favored
            while (favouredthread == 1);
            th2wantstoenter = true;
        }
    }
    // critical section
    // favour the 1st thread
    favouredthread = 1;
    // exit section
    // indicate th2 has completed

```

```

// its critical section
th2wantstoenter = false;
// remainder section
} while (completed == false)
}

```

Peterson's Solution

- A classic software-based solution to the critical-section problem known as Peterson's solution.
- Because of the way modern computer architectures perform basic machine-language instructions, such as load and store, there are no guarantees that Peterson's solution will work correctly on such architectures.
- However the solution provides a good algorithmic description of solving the critical-section problem and addresses the requirements of mutual exclusion, progress, and bounded waiting.

Peterson's solution is restricted to two processes.

- The processes are numbered P_0 and P_1 .
- For convenience, when presenting P_i , we use P_j to denote the other process; that is, $j = 1 - i$.
- Peterson's solution requires the two processes to share two variables:

```
int turn;
```

```
boolean flag[2];
```

The variable `turn` indicates whose turn it is to enter its critical section. – That is, if `turn == i`, then process P_i is allowed to execute in its critical section.

- The flag array is used to indicate if a process is interested to enter its critical section. – That is, if `flag[i]` is true, this value indicates that P_i is ready to enter its critical section.
- The shared variables `flag[0]` and `flag[1]` are initialized to FALSE because neither process is yet interested in the critical section. The shared variable `turn` is set to either 0 or 1 randomly (or it can always be set to say 0).

```
do
```

```
{ flag [ i ] = TRUE ;
```

```
turn = j ;
```

```
while ( flag [ j ] && turn == j )
```

```
{ } // wait
```

```
critical section
```

```
flag [ i ] = FALSE ;
```

```
remainder section
```

```
} while ( TRUE ) ;
```

Explanation

```
bool flag[2] = {false, false};
int turn;
```

```
P0:    flag[0] = true;
P0_gate: turn = 1;
        while (flag[1] && turn == 1)
        {
            // busy wait
        }
        // critical section
        ...
        // end of critical section
        flag[0] = false;
```

```
P1:    flag[1] = true;
P1_gate: turn = 0;
        while (flag[0] && turn == 0)
        {
            // busy wait
        }
        // critical section
        ...
        // end of critical section
        flag[1] = false;
```

- The eventual value of **turn** determines which of the two processes is allowed to enter its critical section first.

We now prove that this solution is correct. We need to show that:

1. Mutual exclusion is preserved.
2. The progress requirement is satisfied.
3. The bounded-waiting requirement is met.

Mutual exclusion

- we note that each P_i enters its critical section only if either **flag [j] == false** or **turn == i**.
- Also note that, if both processes can be executing in their critical sections at the same time, then **flag [0] == flag [1] == true**.
- These two observations imply that P_0 and P_1 could not have successfully executed their **while statements** at about the same time, since the value of **turn** can be **either 0 or 1 but cannot be both**.
- Hence, one of the processes -say, P_j -must have successfully executed the while statement, whereas P_i had to execute at least one additional statement ("**turn==j**").
- However, at that time, **flag [j] == true and turn == j**, and this condition will persist as long as P_j is in its critical section; as a result, mutual exclusion is preserved.

Progress

- Progress is defined as the following: if no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes that are not executing in their remainder sections can participate in making the decision as to which process will enter its critical section next. This selection cannot be postponed indefinitely.

- A process cannot immediately re-enter the critical section if the other process has set its flag to say that it would like to enter its critical section.

Bounded waiting

- Bounded waiting, or bounded bypass means that the number of times a process is bypassed by another process after it has indicated its desire to enter the critical section is bounded by a function of the number of processes in the system.
- In Peterson's algorithm, a process will never wait longer than one turn for entrance to the critical section: After giving priority to the other process, this process will run to completion and set its flag to 1, thereby never allowing the other process to enter the critical section.

Busy Waiting

- The repeated execution of a loop of code while waiting for an event to occur is called busy-waiting.
- The CPU is not engaged in any real productive activity during this period, and the process does not progress toward completion.
- Busy-waiting, busy-looping or spinning is a technique in which a process repeatedly checks to see if a condition is true, such as whether keyboard input or a lock is available.

Busy waiting means a process simply spins, (does nothing but continue to test its entry condition) while it is waiting to enter its critical section. This continues to use (waste)

Semaphore

(Software Based Solution)

- The hardware-based solutions to the critical section problem are complicated for application programmers to use.
- To overcome this difficulty, we can use a synchronization tool called **Semaphore**.
- Definition: A **semaphore S** is an integer variable that, apart from initialization, is accessed only through two standard atomic operations:
 1. **wait** (or **acquire**)
 2. **signal** (or **release**)
- All modifications to the integer value of the semaphore in the wait () and signal() operations must be executed indivisibly.
- That is, when one process modifies the semaphore value, no other process can simultaneously modify that same semaphore value.
- A resource such as a shared data structure is protected by a **semaphore**. You must **acquire** the semaphore before using the resource and **release** the semaphore when you are done with the shared resource.

Wait for Operation

This type of semaphore operation helps you to control the entry of a task into the critical section. However, If the value of wait is positive, then the value of the wait argument X is decremented. In the case of negative or zero value, no operation is executed. It is also called P(S) operation.

After the semaphore value is decreased, which becomes negative, the command is held up until the required conditions are satisfied.

Copy CodeP(S)

```
{
    while (S<=0);
    S--;
}
```

Signal operation

This type of Semaphore operation is used to control the exit of a task from a critical section. It helps to increase the value of the argument by 1, which is denoted as V(S).

Copy CodeP(S)

```
{
    while (S>=0);
    S++;
}
```

Characteristic of Semaphore

Here, are characteristic of a semaphore:

- It is a mechanism that can be used to provide synchronization of tasks.
- It is a low-level synchronization mechanism.
- Semaphore will always hold a non-negative integer value.
- Semaphore can be implemented using test operations and interrupts, which should be executed using file descriptors.

Types of Semaphores

The two common kinds of semaphores are

- Counting semaphores
- Binary semaphores.

Binary Semaphores

The binary semaphores are quite similar to counting semaphores, but their value is restricted to 0 and 1. In this type of semaphore, the wait operation works only if semaphore = 1, and the signal operation succeeds when semaphore= 0. It is easy to implement than counting semaphores.

On some systems, binary semaphores are known as mutex locks, as they are locks that provide mutual exclusion. We can use binary semaphores to deal with the critical-section problem for multiple processes. The n processes share a semaphore, mutex, initialized to 1.

Implementation

```

do {
wait (mutex) ;
// critical section
signal(mutex);
// remainder section
} while (TRUE);

```

Mutual-exclusion implementation with semaphores (Process P_i)

Counting semaphores:

Counting semaphores can be used to control access to a given resource consisting of a finite number of instances.

- The semaphore is initialized to the number of resources available.
- Each process that wishes to use a resource performs a wait() operation on the semaphore (thereby decrementing the count). When a process releases a resource, it performs a signal() operation (incrementing the count).
- When the count for the semaphore goes to 0, all resources are being used. After that, processes that wish to use a resource will block until the count becomes greater than 0.
- The main disadvantage of the semaphore definition given here is that it requires busy waiting.
- While a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the entry code.
- This continual looping is clearly a problem in a real multiprogramming system, where a single CPU is shared among many processes.
- Busy waiting wastes CPU cycles that some other process might be able to use productively. This type of semaphore is also called a spinlock because the process "spins" while waiting for the lock.
- To overcome the need for busy waiting, we can modify the definition of the wait() and signal() semaphore operations.
- When a process executes the wait() operation and finds that the semaphore value is not positive, it must wait.
- However, rather than engaging in busy waiting, the process can block itself. The block operation places a process into a waiting queue associated with the semaphore, and the state of the process is switched to the waiting state.
- Then control is transferred to the CPU scheduler, which selects another process to execute.
- A process that is blocked, waiting on a semaphore S, should be restarted when some other process executes a signal() operation.
- The process is restarted by a wakeup() operation, which changes the process from the waiting state to the ready state. The process is then placed in the ready queue.

- To implement semaphores under this definition, we define a semaphore as a "C" struct:

```

typedef struct {
    int value;
    struct process *list;
} semaphore;

```

The wait() semaphore operation can now be defined as

```

wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();
    }
}

```

- The signal() semaphore operation can now be defined as

```

signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);
    }
}

```

- The block() operation suspends the process that invokes it. The wakeup(P) operation resumes the execution of a blocked process P.
- These two operations are provided by the operating system as basic system calls.
- Note that in this implementation, semaphore values may be negative, although semaphore values are never negative under the classical definition of semaphores with busy waiting.

Counting Semaphore vs. Binary Semaphore

Here, are some major differences between counting and binary semaphore:

Counting Semaphore	Binary Semaphore
No mutual exclusion	Mutual exclusion
Any integer value	Value only 0 and 1
More than one slot	Only one slot
Provide a set of Processes	It has a mutual exclusion mechanism.

Difference between Semaphore vs. Mutex

Parameters	Semaphore	Mutex
Mechanism	It is a type of signaling mechanism.	It is a locking mechanism.
Data Type	Semaphore is an integer variable.	Mutex is just an object.
Modification	The wait and signal operations can modify a semaphore.	It is modified only by the process that may request or release a resource.
Resource management	If no resource is free, then the process requires a resource that should execute wait operation. It should wait until the count of the semaphore is greater than 0.	If it is locked, the process has to wait. The process should be kept in a queue. This needs to be accessed only when the mutex is unlocked.
Thread	You can have multiple program threads.	You can have multiple program threads in mutex but not simultaneously.
Ownership	Value can be changed by any process releasing or obtaining the resource.	Object lock is released only by the process, which has obtained the lock on it.
Types	Types of Semaphore are counting semaphore and binary semaphore and	Mutex has no subtypes.
Operation	Semaphore value is modified using wait () and signal () operation.	Mutex object is locked or unlocked.

Resources Occupancy	It is occupied if all resources are being used and the process requesting for resource performs wait () operation and blocks itself until semaphore count becomes >1.	In case if the object is already locked, the process requesting resources waits and is queued by the system before lock is released.
---------------------	---	--

Advantages of Semaphores:

Here, are pros/benefits of using Semaphore:

- It allows more than one thread to access the critical section
- Semaphores are machine-independent.
- Semaphores are implemented in the machine-independent code of the microkernel.
- They do not allow multiple processes to enter the critical section.
- As there is busy waiting in semaphore, there is never a wastage of process time and resources.
- They are machine-independent, which should be run in the machine-independent code of the microkernel.
- They allow flexible management of resources.

Disadvantage of semaphores

Here, are cons/drawback of semaphore

- One of the biggest limitations of a semaphore is priority inversion.
- The operating system has to keep track of all calls to wait and signal semaphore.
- Their use is never enforced, but it is by convention only.
- In order to avoid deadlocks in semaphore, the Wait and Signal operations require to be executed in the correct order.
- Semaphore programming is a complicated, so there are chances of not achieving mutual exclusion.
- It is also not a practical method for large scale use as their use leads to loss of modularity.
- Semaphore is more prone to programmer error.
- It may cause deadlock or violation of mutual exclusion due to programmer error.

Some Operations related to Semaphore

- There are three algorithms in the hardware approach of solving Process Synchronization problem:
- 1)Test and Set 2) Swap 3) Unlock and Lock
- Hardware instructions in many operating systems help in effective solution of critical section problems.

1. Test and Set :

Here, the shared variable is lock which is initialized to false. TestAndSet(lock) algorithm works in this way – it always returns whatever value is sent to it and sets lock to true. The first process will enter the critical section at once as TestAndSet(lock) will return false and it'll break out of the while loop. The other processes cannot enter now as lock is set to true and so the while loop continues to be true. Mutual exclusion is ensured. Once the first process gets out of the critical section, lock is changed to false. So, now the other processes can enter one by one. Progress is also ensured. However, after the first process any process can go in. There is no queue maintained, so any new process that finds the lock to be false again, can enter. So bounded waiting is not ensured.

Test and Set Pseudocode –

```
//Shared variable lock initialized to false
boolean lock;
boolean TestAndSet (boolean &target)
{  boolean rv = target;
  target = true;
  return rv;
}while(1)
{  while (TestAndSet(lock));
critical section

lock = false;

remainder section}
```

2. Swap :

Swap algorithm is a lot like the Test And Set algorithm. Instead of directly setting lock to true in the swap function, key is set to true and then swapped with lock. So, again, when a process is in the critical section, no other process gets to enter it as the value of lock is true. Mutual exclusion is ensured. Again, out of the critical section, lock is changed to false, so any process finding it gets enter the critical section. Progress is ensured. However, again bounded waiting is not ensured for the very same reason.

Swap Pseudocode –

```
// Shared variable lock initialized to false    // and individual key initialized to false;
boolean lock; Individual key;
void swap(boolean &a, boolean &b)
{  boolean temp = a;
  a = b;      b = temp;
}while (1) {  key = true;
while(key)
  swap(lock,key);
critical section
lock = false;
remainder section }
```

3. Unlock and Lock :

Unlock and Lock Algorithm uses TestAndSet to regulate the value of lock but it adds another value, waiting[i], for each process which checks whether or not a process has been waiting. A ready queue is maintained with respect to the process in the critical section. All the processes coming in next are added to the ready queue with respect to their process number, not necessarily sequentially. Once the ith process gets out of the critical section, it does not turn lock to false so that any process can avail the critical section now, which was the problem with the previous algorithms. Instead it checks if there is any process waiting in the queue. The queue is taken to be a circular queue. j is considered to be the next process in line and the while loop checks from jth process to the last process and again from 0 to (i-1)th process if there is any process waiting to access the critical section. If there is no process waiting then the lock value is changed to false and any process which comes next can enter the critical section. If there is, then that process' waiting value is turned to false, so that the first while loop becomes false and it can enter the critical section. This ensures bounded waiting. So the problem of process synchronization can be solved through this algorithm.

Unlock and Lock Pseudocode –

```
// Shared variable lock initialized to false    // and individual key initialized to false
boolean lock;
Individual key;
Individual waiting[i];
```

```

while(1){
waiting[i] = true;
key = true;
while(waiting[i] && key)
key = TestAndSet(lock);
critical section
j = (i+1) % n;
while(j != i && !waiting[j])
j = (j+1) % n;
if(j == i)
lock = false;
else
waiting[j] = false;
remainder section}

```

Classical problem in Concurrency (Dining Philosophers Problem):

The dining philosophers problem is another classic synchronization problem which is used to evaluate situations where there is a need of allocating multiple resources to multiple processes.

What is the Problem Statement?

Consider there are five philosophers sitting around a circular dining table. The dining table has five chopsticks and a bowl of rice in the middle as shown in the below figure.

At any instant, a philosopher is either eating or thinking. When a philosopher wants to eat, he uses two chopsticks - one from their left and one from their right. When a philosopher wants to think, he keeps down both chopsticks at their original place.

```

while(TRUE)
{
wait(stick[i]);

/*
mod is used because if i=5, next
chopstick is 1 (dining table is circular)
*/

wait(stick[(i+1) % 5]);

/* eat */
signal(stick[i]);

signal(stick[(i+1) % 5]);

/* think */
}

```

When a philosopher wants to eat the rice, he will wait for the chopstick at his left and picks up that chopstick. Then he waits for the right chopstick to be available, and then picks it too. After eating, he puts both the chopsticks down.

But if all five philosophers are hungry simultaneously, and each of them pickup one chopstick, then a deadlock situation occurs because they will be waiting for another chopstick forever. The possible solutions for this are:

- A philosopher must be allowed to pick up the chopsticks only if both the left and right chopsticks are available.
- Allow only four philosophers to sit at the table. That way, if all the four philosophers pick up four chopsticks, there will be one chopstick left on the table. So, one philosopher can start eating and eventually, two chopsticks will be available. In this way, deadlocks can be avoided .

Readers Writer Problem:

Readers writer problem is another example of a classic synchronization problem. There are many variants of this problem, one of which is examined below.

The Problem Statement

There is a shared resource which should be accessed by multiple processes. There are two types of processes in this context. They are **reader** and **writer**. Any number of **readers** can read from the shared resource simultaneously, but only one **writer** can write to the shared resource. When a **writer** is writing data to the resource, no other process can access the resource. A **writer** cannot write to the resource if there are non zero number of readers accessing the resource at that time.

The Solution

From the above problem statement, it is evident that readers have higher priority than writer. If a writer wants to write to the resource, it must wait until there are no readers currently accessing that resource.

Here, we use one **mutex** m and a **semaphore** w. An integer variable read_count is used to maintain the number of readers currently accessing the resource. The variable read_count is initialized to 0. A value of 1 is given initially to m and w.

Instead of having the process to acquire lock on the shared resource, we use the mutex m to make the process to acquire and release lock whenever it is updating the read_count variable.

The code for the **writer** process looks like this:

```
while(TRUE)
{
    wait(w);

    /* perform the write operation */

    signal(w);
}
```

And, the code for the **reader** process looks like this:

```
while(TRUE)
{
    //acquire lock
    wait(m);
    read_count++;
```

```

if(read_count == 1)
    wait(w);

//release lock
signal(m);

/* perform the reading operation */

// acquire lock
wait(m);
read_count--;
if(read_count == 0)
    signal(w);

// release lock
signal(m);
}

```

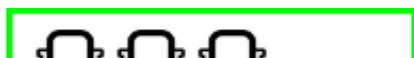
Here is the Code uncoded(explained)

- As seen above in the code for the writer, the writer just waits on the **w** semaphore until it gets a chance to write to the resource.
- After performing the write operation, it increments **w** so that the next writer can access the resource.
- On the other hand, in the code for the reader, the lock is acquired whenever the **read_count** is updated by a process.
- When a reader wants to access the resource, first it increments the **read_count** value, then accesses the resource and then decrements the **read_count** value.
- The semaphore **w** is used by the first reader which enters the critical section and the last reader which exits the critical section.
- The reason for this is, when the first readers enters the critical section, the writer is blocked from the resource. Only new readers can access the resource now.
- Similarly, when the last reader exits the critical section, it signals the writer using the **w** semaphore because there are zero readers now and a writer can have the chance to access the resource.

Sleeping Barber problem:

Problem : The analogy is based upon a hypothetical barber shop with one barber. There is a barber shop which has one barber, one barber chair, and n chairs for waiting for customers if there are any to sit on the chair.

- If there is no customer, then the barber sleeps in his own chair.
- When a customer arrives, he has to wake up the barber.
- If there are many customers and the barber is cutting a customer's hair, then the remaining customers either wait if there are empty chairs in the waiting room or they leave if no chairs are empty.



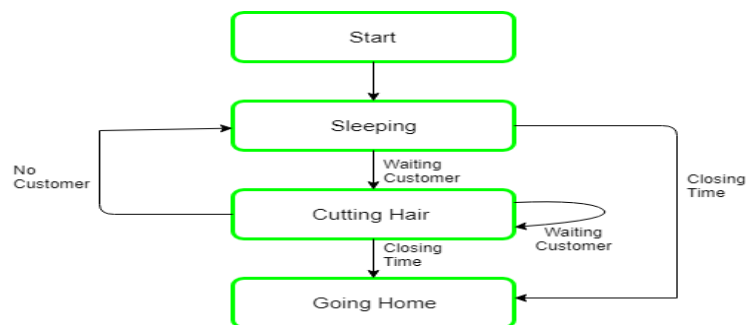
Solution : The solution to this problem includes three semaphores. First is for the customer which counts the number of customers present in the waiting room (customer in the barber chair is not included because he is not waiting). Second, the barber 0 or 1 is used to tell whether the barber is idle or is working, And the third mutex is used to provide the mutual exclusion which is required for the process to execute. In the solution, the customer has the record of the number of customers waiting in the waiting room if the number of customers is equal to the number of chairs in the waiting room then the upcoming customer leaves the barbershop.

When the barber shows up in the morning, he executes the procedure barber, causing him to block on the semaphore customers because it is initially 0. Then the barber goes to sleep until the first customer comes up.

When a customer arrives, he executes customer procedure the customer acquires the mutex for entering the critical region, if another customer enters thereafter, the second one will not be able to anything until the first one has released the mutex. The customer then checks the chairs in the waiting room if waiting customers are less then the number of chairs then he sits otherwise he leaves and releases the mutex.

If the chair is available then customer sits in the waiting room and increments the variable waiting value and also increases the customer's semaphore this wakes up the barber if he is sleeping.

At this point, customer and barber are both awake and the barber is ready to give that person a haircut. When the haircut is over, the customer exits the procedure and if there are no customers in waiting room barber sleeps.



Algorithm for Sleeping Barber problem:

```

Semaphore Customers = 0;
Semaphore Barber = 0;
Mutex Seats = 1;
int FreeSeats = N;

Barber {
    while(true) {
        /* waits for a customer (sleeps). */
        down(Customers);

        /* mutex to protect the number of available seats.*/

```

```

down(Seats);

/* a chair gets free.*/
FreeSeats++;

/* bring customer for haircut.*/
up(Barber);
/* release the mutex on the chair.*/
up(Seats);
/* barber is cutting hair.*/
}}

```

```

Customer {
while(true) {
/* protects seats so only 1 customer tries to sit
in a chair if that's the case.*/
down(Seats); //This line should not be here.
if(FreeSeats > 0) {

/* sitting down.*/
FreeSeats--;

/* notify the barber. */
up(Customers);

/* release the lock */
up(Seats);

/* wait in the waiting room if barber is busy. */
down(Barber);
// customer is having hair cut
} else {
/* release the lock */
up(Seats);
// customer leaves
} } }

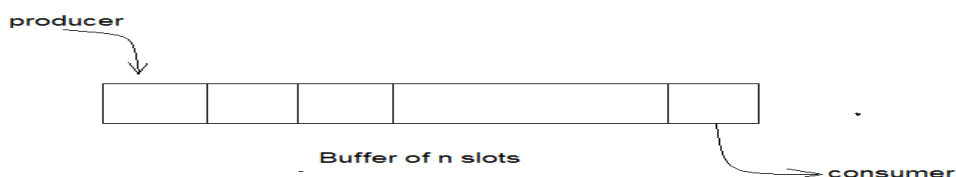
```

Bounded Buffer Problem

Bounded buffer problem, which is also called **producer consumer problem**, is one of the classic problems of synchronization. Let's start by understanding the problem here, before moving on to the solution and program code.

Problem Statement

There is a buffer of n slots and each slot is capable of storing one unit of data. There are two processes running, namely, **producer** and **consumer**, which are operating on the buffer.



A producer tries to insert data into an empty slot of the buffer. A consumer tries to remove data from a filled slot in the buffer. As you might have guessed by now, those two processes won't produce the expected output if they are being executed concurrently.

There needs to be a way to make the producer and consumer work in an independent manner.

Here's a Solution

One solution of this problem is to use semaphores. The semaphores which will be used here are:

- **m**, a **binary semaphore** which is used to acquire and release the lock.
- **empty**, a **counting semaphore** whose initial value is the number of slots in the buffer, since, initially all slots are empty.
- **full**, a **counting semaphore** whose initial value is 0.

At any instant, the current value of **empty** represents the number of empty slots in the buffer and **full** represents the number of occupied slots in the buffer.

The Producer Operation

The pseudocode of the producer function looks like this:

```
do
{
    // wait until empty > 0 and then decrement 'empty'
    wait(empty);
    // acquire lock
    wait(mutex);

    /* perform the insert operation in a slot */

    // release lock
    signal(mutex);
    // increment 'full'
    signal(full);
}
while(TRUE)
```

- Looking at the above code for a producer, we can see that a producer first waits until there is atleast one empty slot.
- Then it decrements the **empty** semaphore because, there will now be one less empty slot, since the producer is going to insert data in one of those slots.
- Then, it acquires lock on the buffer, so that the consumer cannot access the buffer until producer completes its operation.
- After performing the insert operation, the lock is released and the value of **full** is incremented because the producer has just filled a slot in the buffer.

The Consumer Operation

The pseudocode for the consumer function looks like this:

```
do
{
    // wait until full > 0 and then decrement 'full'
    wait(full);
    // acquire the lock
    wait(mutex);
    /* perform the remove operation in a slot */
    // release the lock
    signal(mutex);
    // increment 'empty'
    signal(empty);
}
while(TRUE);
```

- The consumer waits until there is at least one full slot in the buffer.

- Then it decrements the **full** semaphore because the number of occupied slots will be decreased by one, after the consumer completes its operation.
- After that, the consumer acquires lock on the buffer.
- Following that, the consumer completes the removal operation so that the data from one of the full slots is removed.
- Then, the consumer releases the lock.
- Finally, the **empty** semaphore is incremented by 1, because the consumer has just removed data from an occupied slot, thus making it empty.

Bakery Algorithm:

This algorithm is used for solving the critical section problem for n processes.

It is based on a scheduling algorithm used in Bakeries., ice-cream stores. Deli counters, motor vehicle registration and other locations where order must be made out of chaos.

On entering the store, each customer receives number. The customer with the lowest number is served next.

The bakery algorithm cannot guarantee that two processes(customers) do not receive the same number.

In the case of tie the process with the lowest name is served first that is if P_i receive the same number and if $i < j$ then P_i is served first.

The common data structures are--

Int number[n]

Boolean choosing[n];

- Initially

Int number[n] = 0;

Boolean choosing[n] = false;

- **Structure of process P_i**

Do {

Choosing[i] = true;

Number[i] = max(number[0], number[1],.....,number[n-1])+1

Choosing[i] = false;

For(j=0; j<n; j++) {

while(choosing[j];

while((number[j]!=0) && (number[j,j] < number[i,i]));

}

// Critical section

number[i] = 0;

//remainder section

}while(true);

It is now simple to show that mutual exclusion is observed. Consider P_i in its critical section and P_k trying to enter the critical section. When process P_k executes the second while statement for $j=i$, it finds that

number[i] != 0

(number[i,i] < (number[k,k]))

Thus it continues looping in the while statement until P_i leaves the critical section.

If we wish to show that the progress and bounded – waiting requirements are preserved and that the algorithm ensures fairness it is sufficient to observe that the process enter their critical section on a first come first served basis.

Inter process Communication

Inter process communication is the mechanism provided by the operating system that allows processes to communicate with each other. This communication could involve a process letting another process know that some event has occurred or transferring of data from one process to another.

A diagram that illustrates inter process communication is as follows –



The models of interprocess communication are as follows –

Shared Memory Model

Shared memory is the memory that can be simultaneously accessed by multiple processes. This is done so that the processes can communicate with each other. All POSIX systems, as well as Windows operating systems use shared memory.

Advantage of Shared Memory Model

Memory communication is faster on the shared memory model as compared to the message passing model on the same machine.

Disadvantages of Shared Memory Model

Some of the disadvantages of shared memory model are as follows –

All the processes that use the shared memory model need to make sure that they are not writing to the same memory location.

Shared memory model may create problems such as synchronization and memory protection that need to be addressed.

Message Passing Model

Multiple processes can read and write data to the message queue without being connected to each other. Messages are stored on the queue until their recipient retrieves them. Message queues are quite useful for inter process communication and are used by most operating systems.

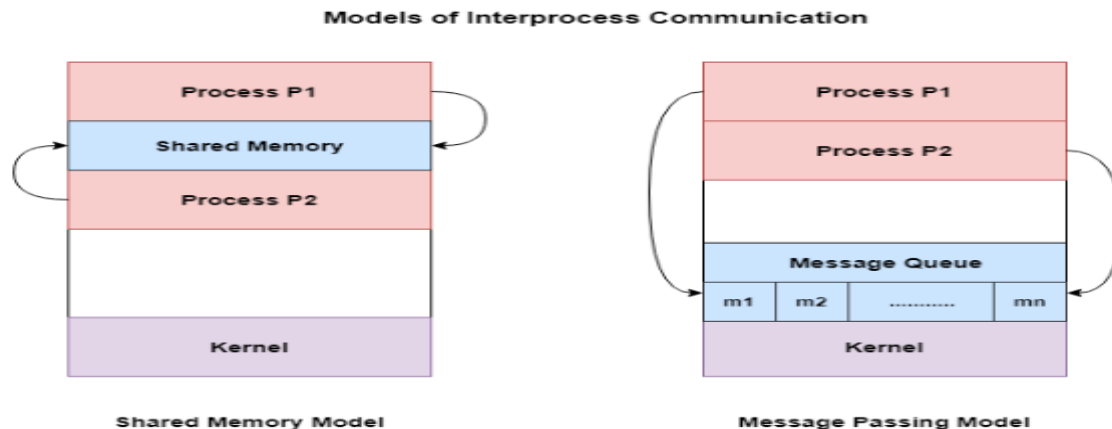
Advantage of Messaging Passing Model

The message passing model is much easier to implement than the shared memory model.

Disadvantage of Messaging Passing Model

The message passing model has slower communication than the shared memory model because the connection setup takes time.

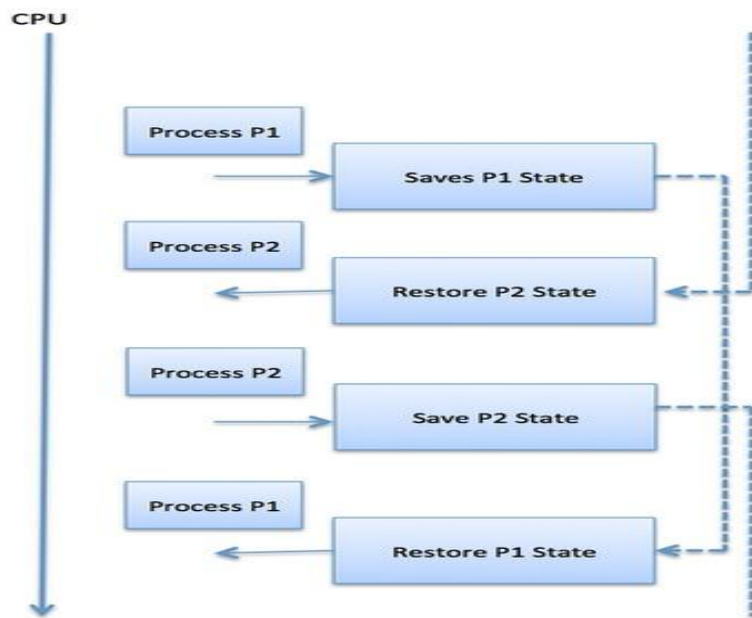
A diagram that demonstrates the shared memory model and message passing model is given as follows –



Context Switch

A context switch is the mechanism to store and restore the state or context of a CPU in Process Control block so that a process execution can be resumed from the same point at a later time. Using this technique, a context switcher enables multiple processes to share a single CPU. Context switching is an essential part of a multitasking operating system features.

When the scheduler switches the CPU from executing one process to execute another, the state from the current running process is stored into the process control block. After this, the state for the process to run next is loaded from its own PCB and used to set the PC, registers, etc. At that point, the second process can start executing.



Context switches are computationally intensive since register and memory state must be saved and restored. To avoid the amount of context switching time, some hardware systems employ two or more sets of processor registers. When the process is switched, the following information is stored for later use.

- Program Counter
- Scheduling information
- Base and limit register value
- Currently used register
- Changed State
- I/O State information
- Accounting information